ToDo

P.

The mousik package*

Celia Rubio Madrigal†

July 1, 2020

Abstract

This is a music-related package for writing musical notation in LaTeX. It is based on the tikz package and includes staves, clefs, key and time signatures, notes and chords, rests, beams and tuplets, accidentals, articulations, slurs and ties, dynamics and hairpins, and piano notation. It uses short commands and allows the user to control the position and shape of the musical elements. Everything is written directly in LaTeX without extra languages or steps.

Keywords

mousik, music, music theory, music notation, music writing, score, sheet, tikz diagram, bar, clef, time signature, note, symbol, rest, dot, chord, accidental, sharp, flat, natural, key

Contents

*This document corresponds to mousik v0.1, dated 2020/07/01.

†Email: celirubio.m@gmail.com

1 Introduction

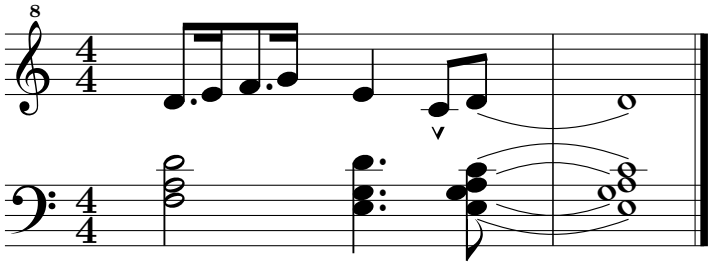
The mousik package has a rather simple idea. It is based on the tikz package, and the score is treated mostly as a drawing. The package does not try to understand the music or decide how everything should be engraved. Instead, it is somewhat similar to "what you see is what you get". It provides short commands for notes, rests, beams, slurs, accidentals, articulations, etc., and lets the user change the position and shape of these elements directly. This makes it useful for small examples, such as those in articles or exercises.

There are several other ways to write music with LaTeX. Some of them use a different program or a different notation language. LilyPond has its own input language and program, and tools such as lilypond-book or LyLuaTeX make it possible to include it in a LaTeX document. The abc package works in a similar way, as the music is written in ABC notation and converted by an external program. MusiXTeX works directly with TeX, but it requires a detailed description of the score and uses a separate pass to calculate horizontal spacing. PMX makes MusiXTeX easier to write, but it uses its own notation and a conversion step.

In comparison, mousik is meant to stay simple. The music is written directly in LaTeX, with short commands and without extra steps.

2 The `mousik` environment

The way to draw a score is by means of the `mousik` environment. For longer examples, open the *examples* document.



```
\begin{mousik}[piano, piano-sep=2]
      \UpperStaff
      \Clef[octave=8] \TimeSig{4}{4}
      \MDots{*/1, , */3, } \Note{|-,=|,|-,=|}{1,2,3,4}
          \Note{4}{2}
      \Artic{^}{0} \Note{8}{0,1}
          \Tie[e=2]{1}
          \Barline
          \Note{1}{1}
      \Barline[t=|.]
```

```
\LowerStaff
\Clef[t=4] \TimeSig{4}{4}
\Note{2}{13/10/8} \Space[1.5]
\MDot{13}\MDot{9}\MDot{7} \Note{4}{13/9/7} \Space[0.5]
\Note{8}{12/10/<9/7}
\Tie[e=2]{12} \Tie[b=0.25,e=1.75]{10}
\Tie[swap,b=0.25,e=1.75]{9} \Tie[swap,e=2]{7}
\Barline
\Note{1}{12/10/<9/7}
 \end{mousik}
```

It has some options:

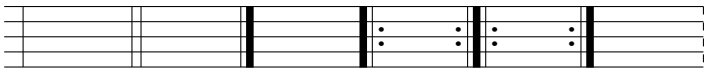
- *color* changes the background color. Default is white.
- *indent* indents the first line. Default is 0, in cm.
- *long* changes the maximum width of the score. Default is 100.
- *hsep* changes the horizontal separation. Default is 8. Beams and parallel dot lists follow the same separation.
- *vsep* changes the vertical separation in case of an overflow. Default is 2, in cm.
- *single* draws a one-line staff, useful for percussion examples. Notes keep the usual absolute height system; height 6 lies on the line.
- *piano* draws an upper and a lower five-line staff. Use `\UpperStaff` and `\LowerStaff` to choose which one receives the following commands. Each staff keeps its own cursor, clef, and key.
- *piano-sep* changes the distance between the two piano staves. Default is 1.6, in cm.
- *piano-bars* chooses whether barlines in piano mode are `joined` across both staves or `separate`. Default is `joined`. The command advances only the active staff.
- *tikz* allows extra options for the outside `tikz` environment.

3 Available commands

3.1 `\Barline`

The `\Barline` command prints a bar on the score. It has some options:

- *t* (type) is the type of bar. Default is `|`, the single bar.
- *xshift* moves the element horizontally (x axis). Default is 0, in cm.
- *yshift* moves the element vertically (y axis). Default is 0, in cm.

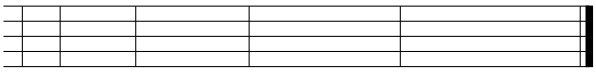


`\Barline t=||  t=|.  t=|:  t=:|:  t=:|  t=:`

Automatic overflow is the safest option after a bar or a space. Otherwise, visual errors may appear (see subsection 4.1). To start a new line explicitly, use `\BreakLine`.

3.2 \Space

The `\Space` command may be used when no bar is needed, or to add extra space between symbols. Its optional argument changes the amount of space. Default is 1, where 1 is the normal separation between symbols. Negative values may also be used (see subsection 4.2).



`[-0.5] [0]  [0.5]  [1]  [1.5]`

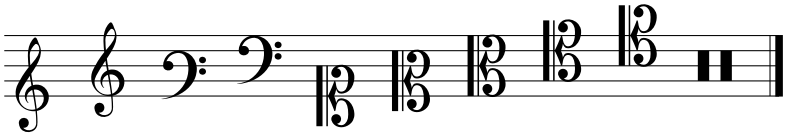
3.3 \BreakLine

The `\BreakLine` command starts a new score line. It resets the horizontal cursor to the beginning of the next line. Unlike an automatic overflow, it does not print a clef or key. When `\BreakLine` is used on the first line, it also determines the staff length of the following lines.

3.4 \Clef

The `\Clef` command may be used at any moment on the score. A clef is also printed automatically after an overflow (see subsection 4.1). It has some options:

- *octave* prints an octave eight on the clef. If it is 8, the eight is printed above the clef; if it is -8, it is printed below the clef. Default is 0.
- *t* (type) is the type of clef. Default is 2 (the Treble Clef).
- *xshift* moves the element horizontally (x axis). Default is 0, in cm.
- *yshift* moves the element vertically (y axis). Default is 0, in cm.



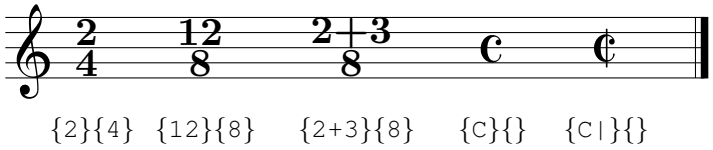
3.5 \TimeSig

The `\TimeSig` command prints a time signature. The first argument is the upper part and the second argument is the lower part. Any number or symbol can be written in them. `C` produces a *common time* symbol, and `C|` produces an *alla breve* symbol.

It has some options:

- *xshift* moves the element horizontally (x axis). Default is 0, in cm.

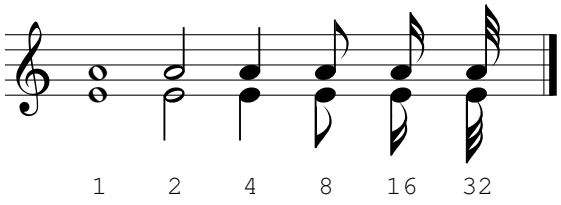
- *yshift* moves the element vertically (y axis). Default is 0, in cm.

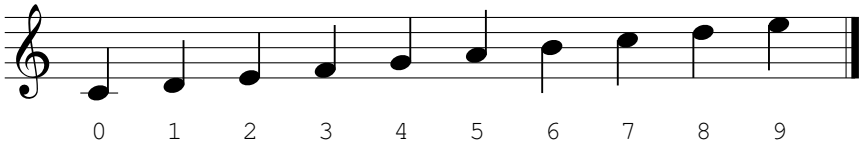


3.6 Notes and rests

3.6.1 \Note

The `\Note` command prints notes and chords. It has two compulsory arguments: the duration and the height. The duration may be 1, 2, 4, 8, 16 or 32, and the height is absolute and in millimeters, starting from the middle C as 0.

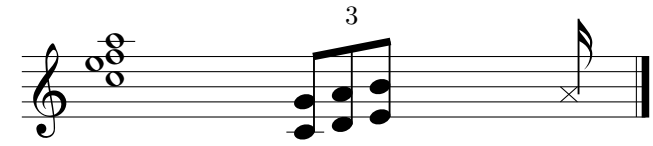




The second argument may also contain heights separated by `/` to form a chord, or positions separated by commas to form a group of notes. Each position in a group may itself be a chord. To move the head of an item in a chord to the left, use `<`.

It has some options:

- *beam* chooses the beam width. Default is 1, in mm.
- *sloped* makes a beam of three or more notes follow a slope from the first note to the last instead of being horizontal. Two-note beams are always oblique.
  1.sloped creates non vertical end of beams
- *head* chooses the type of head. Default is `.` (round), but `x` (cross) and `o` (hollow) are also possible.
- *stem* chooses the stem height. Default is 5, in mm. By default, stems point up below height 6 and down otherwise, but *swap* can reverse this.
- *tuplet* prints the number of a tuplet. The number is calculated by default, but it may also be assigned. *tuplet-bracket* adds a bracket, and *tuplet-xshift* and *tuplet-yshift* move the tuplet marking.
- *xshift* and *yshift* move the element horizontally and vertically. Default is 0, in cm.



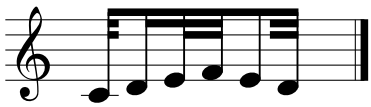

```
\Note
{1}{12/10/<9/7}
```

```
\Note[head=x,stem=7]
{16}{5}
```

```
\Note[sloped,tuplet,swap]
{8}{0/4,1/5,2/6}
```

Notes with different durations may also be joined in the same beam group. In this case the first argument is a list. `-` means an eighth note, `=` a sixteenth note and ≡ a thirty-second note. A `|` may be added before or after a symbol to mark where that level begins or ends. If there is no neighbouring note at that level, the side of the `|` also chooses the direction. The forms `//` and `///` are

equivalent to = and ≡. Inside a beam group, ! leaves a space of 0.75 cm, and an empty item leaves a note position.



```
\Note{|≡,|=,|≡,≡|,-,≡|}{0,1,2,3,2,1}
```

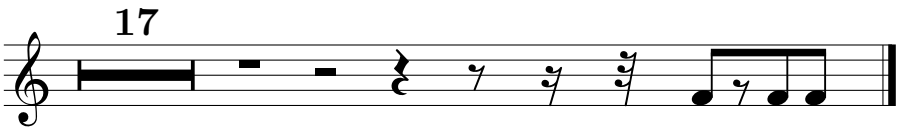
2.the first one looks too short and the last one too long, not half half

3.6.2 \Rest

The \Rest command prints a rest. It accepts its duration as an argument: 1, 2, 4, 8, 16 or 32.

It has some options:

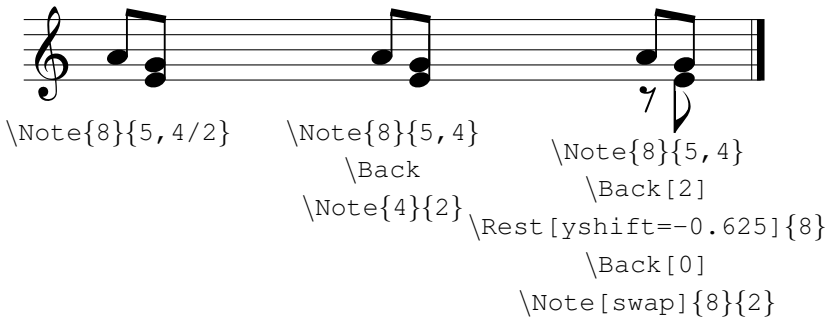
- *multi* transforms the rest into a multimeasure rest, where the rest lasts an arbitrary amount of bars.
- *xshift* moves the element horizontally (x axis). Default is 0, in cm.
- *yshift* moves the element vertically (y axis). Default is 0, in cm.




```
                1    2    4    8   16   32
\Rest[multi]{17}                    \Note{|-, ,-,-|}{3, ,3,3}
                                          \Space[-2]
                                    \Rest[yshift=-0.2]{8}
                                          \Space[1]
```

3.6.3 \Back

The \Back command moves the horizontal cursor backwards. Its optional argument is the number of note positions to step backwards, with 1 as default. It is useful for superimposing independent voices, as well as a manual alternative to chords.



3.why 0.625?4.why is back[0] needed here?5.Looks like space[-2] works better than doing back? perhaps its better to have a separate section on using negative space for rests and polyphony

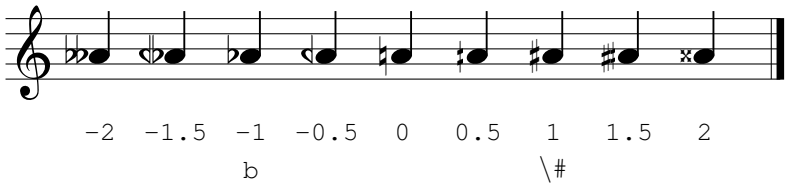
3.7 Keys and accidentals

3.7.1 \Acc

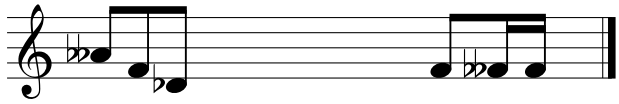
The `\Acc` command prints an accidental before a note. It accepts 2 arguments. The first argument is the type of accidental: a number ranging from -2 to 2. The flat ♭ symbol can also be called with a `b` and the sharp ♯ symbol can be called with `\#`. The second argument is the height of the symbol, which corresponds to those of the notes.

It also has some options:

- *xshift* moves the element horizontally (x axis). Default is 0, in cm.
- *yshift* moves the element vertically (y axis). Default is 0, in cm.



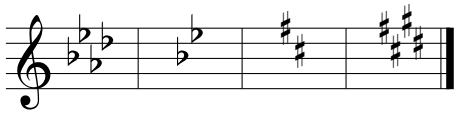
When there are several notes, the `\Accs` command might be useful. It accepts as an argument a list of pairs: type/height. Where there is no accidental, the spot can be left empty. It also accepts the *xshift* and *yshift* options.



```
\Accs{-2/5, ,-1/1}      \Accs{,!,-2/3,}
  \Note{8}{5,3,1}  \Note{|-,!,=,=|}{3,!,3,3}
```

3.7.2 \Key

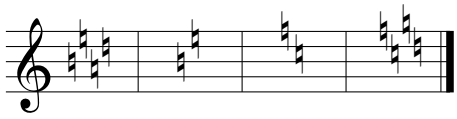
The `\Key` command prints the key at any moment on the score, although it is automatically printed with line overflows (see subsection 4.1). It accepts an argument: a positive whole number from 1 to 7 for sharps ♯, and a negative whole number from -1 to -7 for flats ♭.



```
{-4}    {-2}    {2}    {4}
```

3.7.3 \NoKey

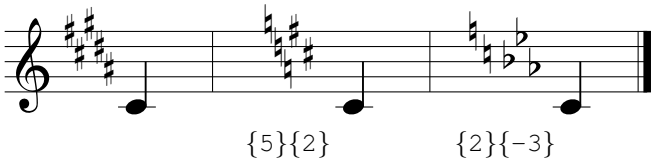
The `\NoKey` command is the same as the `\Key` command, but it prints naturals ♮ instead of sharps or flats.



```
{-4}    {-2}    {2}    {4}
```

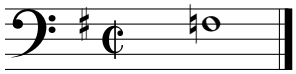
3.7.4 \MixedKey

The `\MixedKey` command prints a change of key signature. It accepts two arguments: the old key signature and the new one. For example, `\MixedKey{5}{2}` changes from five sharps to two sharps, and `\MixedKey{2}{-3}` changes from two sharps to three flats. Cancellation naturals are printed when needed.



```
{5}{2}          {2}{-3}
```

Be careful: the \Key, \NoKey and \MixedKey commands are affected by the current clef. Other commands such as \Note or \Acc are not affected and need absolute height values.



```
\begin{mousik}
    \Clef[t=4]
    \Key{1}
    \TimeSig{C|}{}

    \Acc{0}{8}
    \Note{1}{8}
    \Barline[t=|.]
\end{mousik}
```

3.8 Other symbols

3.8.1 \MDot

6.change name of mdot? to another reasonable thing that is not dot (as this exists already i think) The \MDot command prints one or two dots in front of a note or rest. It accepts a compulsory argument: the height of the dot, which corresponds to those of the notes.

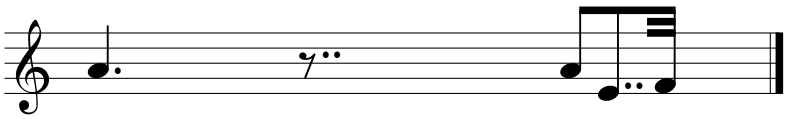
It has some options:

- *t* (type) accepts ** or .. to produce 2 dots. The default is *.

- *xshift* moves the element horizontally (x axis). Default is 0, in cm.

- *yshift* moves the element vertically (y axis). Default is 0, in cm.

When there are several notes, the \MDots command can be useful. It accepts as an argument a list of pairs: symbols/height, where symbols can be * or **. Where there are no dots, the spot can be left empty. It also accepts the *xshift* and *yshift* options.



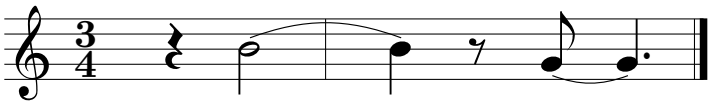
```
\MDot{5}   \MDot[t=**]{7}     \MDots{,**/3,!,}
\Note{4}{5}     \Rest{8}    \Note{|-,-,!,≡|}{5,2,!,3}
```

7.this one is currently ok in terms of = spacing, but if i change it at one point i will need to check

3.8.2 `\Tie`

The `\Tie` command draws a tie between two notes. It has a compulsory argument, the height of both notes, and some options:

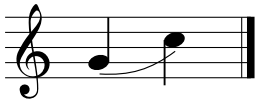
- *angle* changes the angle in which it is drawn. Default is 20, in degrees.
- *b* (beginning) changes the starting point. Default is 0, in cm.
- *e* (end) changes the ending point. Default is 1, in cm.
- *swap* prints the element upside down. Default is false, which means like a *u* if the height is less than 6, and like an *n* otherwise.
- *width* changes the line width. Default is 0.15, in mm.
- *xshift* moves the element horizontally (x axis). Default is 0, in cm.
- *yshift* moves the element vertically (y axis). Default is 0, in cm.



`\Tie[e=2]{6}` `\Tie{4}`

3.8.3 `\Slur`

The `\Slur` command draws a slur between two consecutive notes. It accepts two compulsory arguments: the absolute heights of its beginning and end. It accepts the same options as `\Tie`: *angle*, *b*, *e*, *swap*, *width*, *xshift* and *yshift*.

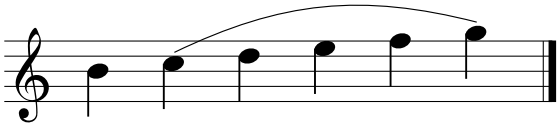


`\Slur{4}{7}`

3.8.4 The `Slurred` environment

The `Slurred` environment draws a slur between the notes placed inside. It has two mandatory arguments: the heights of both beginning and end notes. It accepts the same options as `\Slur`: *angle*, *b*, *e*, *swap*, *width*, *xshift* and *yshift*.

8.angle in and out?



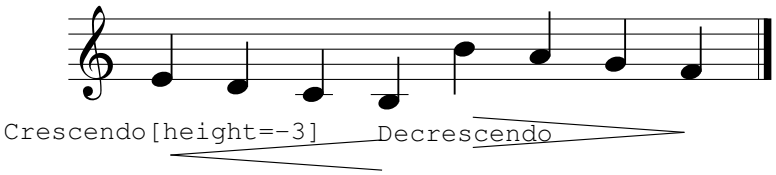
```
\begin {Slurred}{7}{11}
\foreach\i in {7,...,11}{\Note{4}{\i}}
\end {Slurred}
```

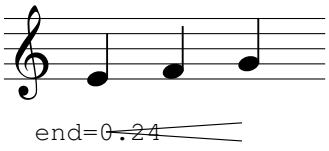
3.8.5 The `Crescendo` and `Decrescendo` environments

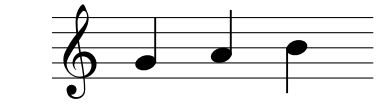
The `Crescendo` and `Decrescendo` environments draw hairpins between the notes placed inside. Their options are:

- *height* moves the centre of the hairpin vertically. Default is 0.
- *begin* and *end* set the full opening of the hairpin at its beginning and end, in cm. A crescendo defaults to `begin=0,end=0.4`. A decrescendo defaults to `begin=0.4,end=0`.
- *b* and *e* move the beginning and ending points horizontally from the first and last heads. Defaults are 0.1 and -0.1, in cm.
- *width* changes the line width. Default is 0.15, in mm.
- *xshift* moves the hairpin horizontally. Default is 0, in cm.
- *yshift* moves the hairpin vertically. Default is 0, in cm.

A hairpin that continues after a line break should be written as two independent pieces. Give the first piece a chosen *end* opening and use the same value as *begin* on the next line. **9.these examples are not so good as they are too many but the line break should still exist**

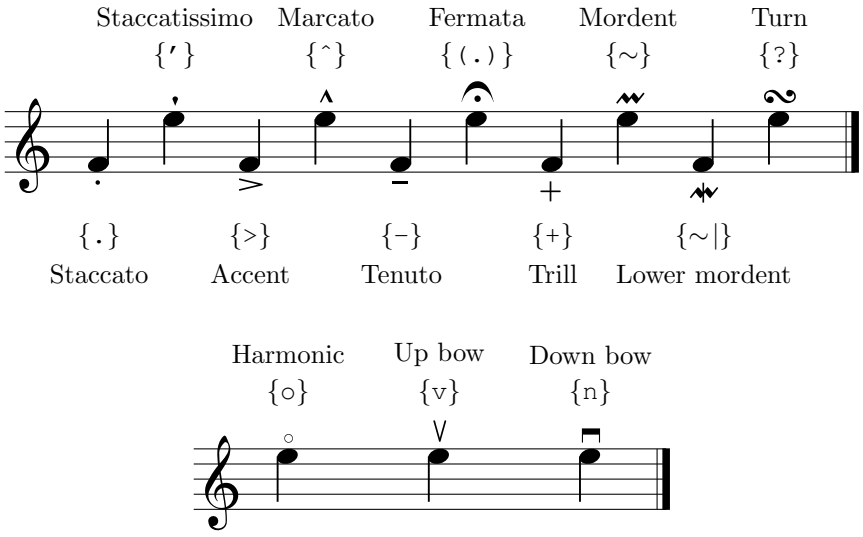






3.8.6 \Artic and \Symbol

The \Artic command prints an articulation symbol on a successive note. It accepts two arguments. The first one is the type of articulation. The second one is the height of the symbol, which corresponds to those of the notes —but it need not be the height of the corresponding note.

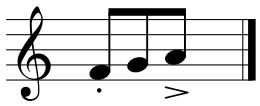


It has some options:

- *xshift* moves the element horizontally (x axis). Default is 0, in cm.

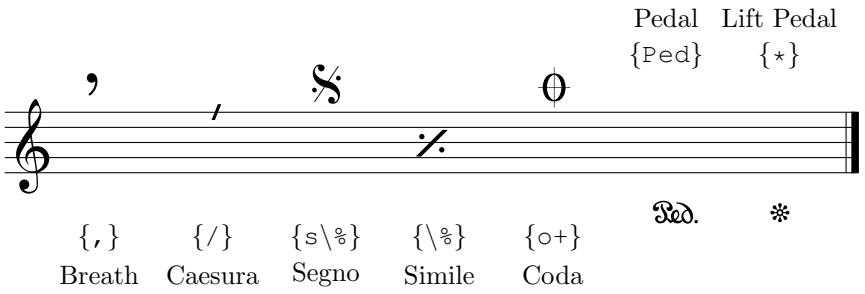
- *yshift* moves the element vertically (y axis). Default is 0, in cm.

When there are several notes, the \Artics command might be useful. It accepts as an argument a list of pairs: symbols/height. Where there are no symbols, the spot shall be left empty. It also accepts the *xshift* and *yshift* options.



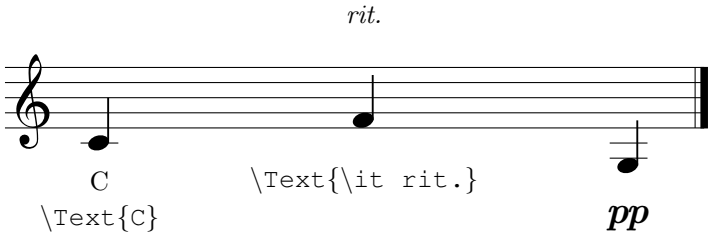
```
\Artics{./3, ,>/3}
  \Note{8}{3,4,5}
```

The `\Symbol` command is similar to the `\Artic` command, but it only accepts the type of symbol and not the height. These are symbols that have a fixed position on the score. It also accepts the *xshift* and *yshift* options.



3.9 \Text and \Dynamics

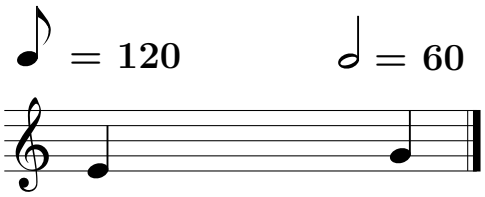
The `\Text` command allows the printing of any text string. It has a compulsory argument: the text to be shown. The `\Dynamics` command works in the same way, but uses the usual font of dynamics. Both commands accept the *xshift* and *yshift* options.



3.10 \Tempo

The `\Tempo` command prints a metronome tempo indicator. It has two arguments: the duration of the note and the numerical tempo which will be printed.

It has the same options as the `\Text` command. An optional *yshift* of 2.5 is recommended.




```
\Tempo[yshift=2.5]{8}{120}
                        \Tempo[yshift=2.4]{2}{60}
```

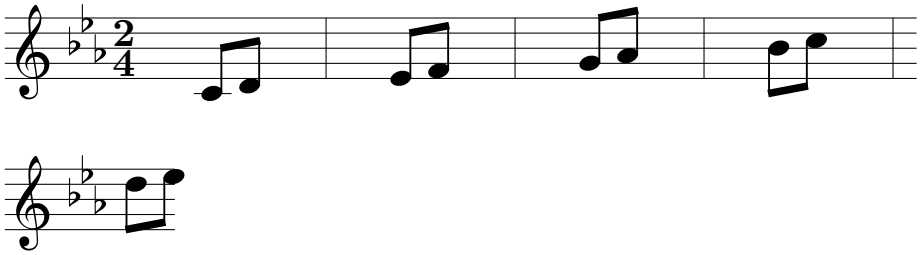
10.for some reason the 2 needs 2.4 instead of 2.5

4 Tips and observations

4.1 Overflow of lines

The environment has a maximum width, which may be changed with the *long* option. When the cursor reaches that width, the next element triggers an automatic overflow. This starts a new staff line and repeats the current clef and key signature.

Sometimes relying on automatic overflow can produce awkward spacing. For a more predictable layout you may use `\BreakLine`, and add or remove spaces for a more controlled positioning.



4.2 How do spaces work?

Spaces serve as the manual progression of an invisible cursor during the printing.

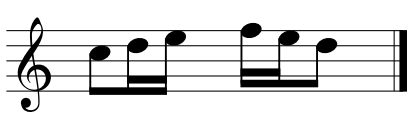
Most elements are separated by a distance of 1 cm, so a `\Space` command that has an argument smaller than -1 will actually step backwards. A `\Space` with a 0 as the argument does nothing.

The `\Space` command is truly relevant in this package. Not a lot of logic is

internally implemented, so the user is supposed to adjust everything at their convenience.

4.3 What if it does become too cumbersome: your own commands

If a concrete rhythm is constantly used on a music piece, but the commands become increasingly long and tedious, the user might want to summarize it by creating their own vocabulary.



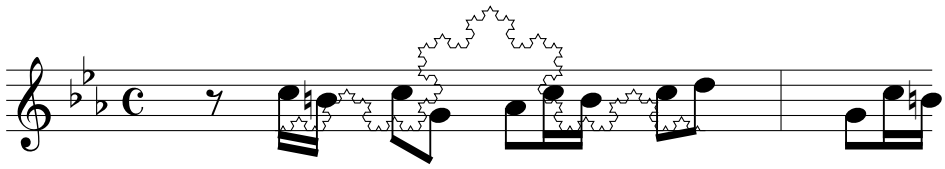
```
\newcommand{\ttc}[2][]{\Note[#1]{|-,|=,=|}{#2}}
\newcommand{\tct}[2][]{\Note[#1]{|=,=|,-|}{#2}}

\begin{mousik}
      \Clef
      \ttc{7,8,9}
      \tct{10,9,8}
      \Barline[t= |.]
\end{mousik}
```

4.4 On your own: working with the tikz package

As stated in the introduction (section 1), the mousik package is based on the tikz package. In fact, the `mousik` environment is a disguised version of a `tikzpicture` environment, but with some variables and a printed staff.

This implies two separate things:

1. The mousik environment admits other commands inside, not just the commands explained in this text. For example, I might draw nodes, lines, or even a fractal behind my score:



2. The commands explained here might be used in a `tikzpicture` environment, and will appear with no staff. However, the command `\Init` should be used at the beginning to avoid initialization errors.